

CHAPTER 03

03. 외부 모듈 — npm 패키지를 직접 설치하며 익히기

Node.js · Express · NestJS 강의 · 2026

앞 장(02)에서는 파일·서버·환경변수를 Node.js에 기본으로 들어 있는 **내장 모듈**로 다뤘다. 설치할 게 없었다. 하지만 실무 코드의 대부분은 남이 미리 만들어 둔 **외부 모듈**, 곧 npm 패키지 위에서 돌아간다. 로깅, 입력 검증, 날짜 계산 같은 일을 매번 직접 짜는 사람은 없다. 이미 검증된 패키지를 가져다 쓴다.

이 장은 그 "가져다 쓰는 흐름"을 손에 익히는 장이다. `npm install` 로 패키지를 받고, `require` 로 불러오고, 문서대로 호출한다. 다루는 패키지는 다섯이다 — **dotenv**(환경변수), **winston**(로깅), **validator**(검증), **uuid**(고유 ID), **dayjs**(날짜). 어느 것 하나 거창하지 않다. 작은 도구 다섯 개를 직접 설치해 보면서, npm 생태계가 어떻게 돌아가는지 몸으로 익히는 게 목표다.

이 자료는 **개념 연습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- `npm install <패키지>` 가 `package.json` 의 `dependencies` 를 채우는 과정
- `.env` 파일로 설정값을 코드 바깥에 빼두는 법 (dotenv)
- 로그를 레벨로 나눠 콘솔과 파일에 동시에 남기는 법 (winston)
- 이메일·URL·비밀번호 같은 입력값을 검증하는 법 (validator)
- 충돌하지 않는 고유 ID를 만드는 법 (uuid)
- 날짜를 더하고·빼고·비교하고·포맷하는 법 (dayjs, 플러그인 포함)

내용은 다섯 주제로 나뉜다. 각 주제는 작은 도구 하나를 직접 설치해 콘솔 출력을 눈으로 확인하는 식으로 익힌다.

주제	핵심
dotenv	<code>.env</code> 의 값을 <code>process.env</code> 로 읽기
winston	레벨별 로깅, 콘솔+파일 출력
validator	이메일·URL·비밀번호 형식 검증
uuid	충돌하지 않는 고유 ID 생성
dayjs	날짜 포맷·연산·비교

핵심 개념 — 패키지를 직접 설치한다는 것

내장 모듈은 `require("fs")` 처럼 그냥 불러 쓰면 됐다. 외부 모듈은 한 단계가 더 있다. **먼저 내려받아야** `require` 로 부를 수 있다. 그 내려받는 명령이 `npm install` 이다.

```
npm install dotenv
```

이 한 줄이 세 가지를 동시에 한다.

1. **node_modules/** 폴더에 dotenv의 실제 코드를 내려받는다.
2. **package.json**의 **dependencies**에 **"dotenv": "^16.4.5"** 같은 줄을 추가한다. "이 프로젝트는 dotenv가 필요하다"는 기록이다.
3. **package-lock.json**에 정확히 어떤 버전을 받았는지를 못 박는다.

이 장 폴더에는 **package.json**이 이미 채워진 채로 동봉돼 있다. 그래서 **npm install**만 치면 다섯 패키지가 한 번에 들어온다. 하지만 이 장의 진짜 목적은 그 자동 설치가 아니라, **명령을 직접 쳐서 package.json이 채워지는 걸 자기 눈으로 보는 것**이다. 강의가 끝나고 자기 프로젝트를 시작할 때 똑같은 명령을 칠 테니까.

dependencies에 적힌 ^16.4.5의 의미 — 캐럿(^)은 "이 버전 이상, 단 메이저 버전(맨 앞 숫자)은 안 올린다"는 뜻이다. ^16.4.5라면 16.4.5부터 17.0.0 직전까지를 허용한다. 새 기능과 버그 수정은 받되, 호환을 깨는 큰 변경은 막겠다는 규칙이다. 실제로 받은 버전을 정확히 고정하는 건 **package-lock.json**의 몫이다.

require VS import — 이 장의 코드는 **require()**로 모듈을 불러온다. **package.json**에 **"type": "commonjs"**가 적혀 있어서다. 이게 Node.js의 전통적인 모듈 방식(CommonJS)이다. **import**를 쓰는 ES 모듈 방식도 있지만, 이 과정에서는 줄곧 **require**로 통일한다.

준비

예제 폴더에는 완성된 **package.json**이 동봉돼 있다. 패키지부터 설치한다.

```
npm install # dotenv, winston, validator, uuid, dayjs가 한 번에 설치된다
```

dotenv 예제는 **.env** 파일을 읽는다. 동봉된 예시 파일을 복사해 만든다.

```
cp .env.example .env
```

각 주제의 코드는 서버가 아니라 **한 번 실행되고 끝나는 스크립트**다. **Ctrl+C**로 멈출 필요 없이, 콘솔에 찍힌 결과를 읽으면 된다. 직접 설치한 명령으로 **package.json**이 채워지는지, 실행 결과가 콘솔에 어떻게 찍히는지를 눈으로 확인하는 게 핵심이다.

환경변수 다루기 (dotenv)

포트 번호, 데이터베이스 이름, API 키. 이런 값을 코드 안에 그대로 박아 두면 두 가지가 곤란하다. 환경마다(내 컴퓨터 / 운영 서버) 값이 다른데 코드를 고쳐야 하고, API 키 같은 민감한 정보가 소스에 그대로 남아 깃에 올라간다. **dotenv**는 이 값들을 `.env` 라는 별도 파일로 빼내, 코드에서는 `process.env` 로 읽게 한다.

먼저 `.env` 파일의 모양을 보자. 이 장에서 쓰는 예시는 이렇게 생겼다.

```
# 서버 설정
PORT=3000                                # 서버가 실행될 포트 번호

# 데이터베이스 설정
DB_NAME=myapp                            # 데이터베이스 이름
...

# API 설정
API_KEY=your_api_key                     # API 키 (실제 키로 변경 필요)

# 환경 설정
NODE_ENV=development                     # development 또는 production
```

키=값 형식의 평범한 텍스트다. 코드가 읽는 키는 `PORT`, `DB_NAME`, `API_KEY`, `NODE_ENV` 넷이다.

```
// .env 파일의 내용을 불러온다
require("dotenv").config();
```

이 한 줄이 `dotenv`의 전부라고 해도 된다. `require("dotenv").config()` 를 호출하면, `dotenv`가 같은 폴더의 `.env` 파일을 읽어 그 안의 키=값 을 전부 `process.env` 에 채워 넣는다. 이 줄을 파일 맨 위에 두는 게 중요하다. 다른 코드가 `process.env` 를 읽기 전에 값이 먼저 들어와 있어야 하기 때문이다.

```
// 환경변수 읽어보기
console.log("서버 포트:", process.env.PORT);
console.log("DB 이름:", process.env.DB_NAME);
console.log("API 키:", process.env.API_KEY);
```

`config()` 가 채워 넣은 값을 `process.env.PORT` 처럼 꺼낸다. `process.env` 는 Node.js가 기본으로 들고 있는 환경 변수 객체이고, `dotenv`는 거기에 `.env` 의 내용을 채워 넣는 역할만 한다. 한 가지 기억할 점 — 여기서 꺼낸 값은 전부 문자열이다. `PORT=3000` 이라고 적어도 `process.env.PORT` 는 숫자 `3000` 이 아니라 문자열 `"3000"` 이다.

```
// 환경변수를 객체로 모아서 사용하기
const myConfig = {
  port: process.env.PORT || 3000, // 기본값 설정 가능
  dbName: process.env.DB_NAME,
  apiKey: process.env.API_KEY,
};

console.log("설정 내용:", {
  ...myConfig,
  apiKey: "****", // 보안을 위해 API 키는 숨김
});
```

흠어진 `process.env` 접근을 한곳에 모은 흔한 패턴이다. 두 군데를 짚어 두자.

- `process.env.PORT || 3000` — `.env` 에 `PORT` 가 없으면 `process.env.PORT` 는 `undefined` 다. 그때 `|| 3000` 이 기본값을 채운다. 설정이 빠져도 코드가 멈추지 않게 하는 안전장치다.
- 로그를 찍을 때 `apiKey: "****"` 로 덮어쓴 부분 — 키 값을 콘솔에 그대로 흘리지 않으려는 습관이다. 민감한 값은 로 그에도 남기지 않는다.

```
// 필수 환경변수 검증하기
if (!process.env.API_KEY) {
  console.error("API_KEY가 설정되지 않았습니다!");
}

// 개발환경/운영환경 구분하기
const isDevelopment = process.env.NODE_ENV === "development";

if (isDevelopment) {
  console.log("개발 환경에서 실행 중입니다");
} else {
  console.log("운영 환경에서 실행 중입니다");
}
```

마지막 두 조각은 `process.env` 를 실제로 활용하는 방법이다. 필수 값이 없으면 곧장 경고를 띄우고(없는 채로 굴리다 한 참 뒤에 터지는 것보다 낫다), `NODE_ENV` 값으로 지금이 개발 환경인지 운영 환경인지 갈라 동작을 바꾼다. `NODE_ENV` 는 Node 생태계 전반에서 환경을 구분하는 사실상의 표준 키다.

.env 는 깃에 올리지 않는다 — `.env` 에는 실제 비밀번호와 키가 들어간다. 그래서 `.gitignore` 에 `.env` 를 넣어 추적에서 빼고, 대신 값을 비운 `** .env.example **`만 깃에 올린다. 새로 합류한 동료는 `.env.example` 을 복사해 자기 값을 채운다. 이게 팀이 설정을 공유하는 표준 방식이다.

제대로 된 로깅 (winston)

지금까지는 `console.log` 로 출력을 봤다. 학습용으로는 충분하지만, 실제 서비스에서는 부족하다. 로그를 **중요도로 나누고**(에러만 따로 보고 싶다), **파일로도 남기고**(서버 콘솔은 닫히면 사라진다), **시각을 붙여야**(언제 일어난 일인지) 한다.

winston은 이 세 가지를 갖춘 로깅 라이브러리다.

```
const winston = require("winston");

// 가장 기본적인 로거 생성
const logger = winston.createLogger({
  level: "info", // info 레벨 이상의 로그만 출력
  format: winston.format.simple(), // 간단한 텍스트 형식
  transports: [
    new winston.transports.Console(), // 콘솔에 출력
    new winston.transports.File({ filename: "app.log" }), // 파일에 저장
  ],
});
```

`winston.createLogger(. )` 로 로거 하나를 만든다. 설정 객체의 세 항목이 핵심이다.

- **level: "info"** — 어느 중요도까지 기록할지 정하는 문턱이다. "info" 로 두면 info와 그보다 중요한 것(warn, error)은 남기고, 덜 중요한 것(debug 등)은 버린다.
- **format** — 로그를 어떤 모양으로 찍을지다. `simple()` 은 `info: 메시지` 정도의 단순한 텍스트다.
- **transports** — 로그를 어디로 내보낼지의 목록이다. `Console()` 은 터미널로, `File({ filename: "app.log" })` 는 파일로 보낸다. 둘을 배열에 함께 넣었으니 한 번 찍으면 콘솔과 파일에 동시에 기록된다. 이렇게 출력 대상을 여러 곳에 한 번에 보내는 것이 winston의 핵심이다.

```
// 로그 레벨 알아보기 (중요도 순)
console.log("로그 레벨: error > warn > info > debug > silly");

logger.error("에러 발생! - 가장 중요한 메시지");
logger.warn("경고! - 주의가 필요한 메시지");
logger.info("정보 - 일반적인 정보 메시지");
logger.debug("디버그 - 개발 중에만 사용하는 메시지");
```

로그 레벨은 **중요도 순서**다. `error` 가 가장 위, `silly` 가 가장 아래다. 위에서 로거를 `level: "info"` 로 만들었으므로, `error · warn · info` 세 줄은 출력되지만 `logger.debug(. )` 는 **화면에 안 나온다**. info보다 덜 중요해서 문턱에 걸린 것이다. 운영 중에는 `level` 을 올려 자잘한 로그를 끄고, 개발 중에는 내려 모두 보는 식으로 조절한다.

```
// 에러 발생 시
try {
  throw new Error("파일을 찾을 수 없습니다");
} catch (error) {
  logger.error("로그인 처리 중 오류 발생", error.message);
}
```

실제 코드에서 로깅이 가장 빛나는 자리다. `try.catch` 로 에러를 붙잡아 `logger.error` 로 남긴다. 콘솔에 흘리고 마는 게 아니라 `app.log` 파일에도 함께 쌓이니, 나중에 "그때 무슨 에러가 났지?"를 파일을 열어 확인한다.

```
// 간단한 로그 포매팅
const simpleLogger = winston.createLogger({
  level: "info",
  format: winston.format.combine(
    winston.format.timestamp(), // 시간 추가
    winston.format.printf(({ timestamp, level, message }) => {
      return `${timestamp} [${level}]: ${message}`;
    })
  ),
  transports: [new winston.transports.Console()],
});

simpleLogger.info("타임스탬프가 포함된 로그");
```

로그 모양을 직접 짜는 방법이다. `format.combine(.)` 은 여러 포맷 규칙을 겹쳐 쌓는다.

- `timestamp()` — 각 로그에 시각 정보를 붙인다.
- `printf(({ timestamp, level, message }) => ..)` — 그 조각들을 받아 한 줄을 직접 조립한다. 여기서는 `2024-. [info]: 메시지` 모양이 된다.

앞의 `simple()` 은 시각이 없었지만, 이렇게 `timestamp()` 를 끼우면 언제 찍힌 로그인지가 남는다. 실서비스 로그에서 시각은 빠질 수 없다.

이 코드를 실행하면 폴더에 `app.log` 가 생긴다. 콘솔에 찍힌 것과 같은 내용이 파일에도 들어 있는지 열어 보자. 여러 번 실행하면 내용이 덮이지 않고 계속 이어 붙는다(`append`). 로그는 원래 쌓아 두는 것이기 때문이다.

`console.log` 로 충분하지 않은가 — 잠깐 값을 찍어 볼 때는 `console.log` 가 빠르고 편하다. 하지만 서비스가 돌기 시작하면 "에러만 따로 모으고", "파일로 보관하고", "시각과 레벨을 붙이는" 일이 필요해진다. `winston`은 그걸 설정 한 번으로 처리한다. `console.log` 와 `winston`은 경쟁 관계가 아니라 쓰임이 다르다.

입력 검증과 고유 ID (validator, uuid)

사용자가 보낸 값은 믿을 수 없다. 이메일 칸에 이메일이 아닌 글자가, 숫자 칸에 문자가 들어올 수 있다. 그 값을 그대로 저장하기 전에 형식이 맞는지 따지는 게 ****검증(validation)****이다. **validator**는 이메일·URL·비밀번호 등 흔한 형식을 검사하는 함수를 모아 둔 패키지다. 함께 등장하는 **uuid**는 충돌하지 않는 고유 ID를 만든다.

```
const validator = require("validator");
const { v4: uuidv4 } = require("uuid");
```

uuid를 불러오는 줄을 눈여겨보자. `require("uuid")` 는 여러 기능을 담은 객체를 돌려준다. `{ v4: uuidv4 }` 는 그중 `v4` 함수만 꺼내 `uuidv4` 라는 이름으로 바꿔 받는 구조분해다. uuid에는 여러 버전(v1, v4 등)이 있는데, 가장 널리 쓰는 게 무작위 기반의 v4다.

validator — 형식 검사

```
// 이메일 검증
console.log("이메일 검증:", validator.isEmail("test@example.com"));

// URL 검증
console.log("URL 검증:", validator.isURL("https://www.example.com"));

// IP 주소 검증
console.log("IP 주소 검증:", validator.isIP("192.168.1.1"));

// 전화번호 검증
console.log("전화번호 검증:", validator.isMobilePhone("010-1234-5678", "ko-KR"));
```

validator의 함수는 거의 다 `is.` 모양이고, ****형식이 맞으면 `true`, 아니면 `false` ****를 돌려준다. 쓰임이 직관적이다.

- `isEmail`, `isURL`, `isIP` — 각각 이메일·URL·IP 형식인지 본다.
- `isMobilePhone("010-.", "ko-KR")` — 두 번째 인자가 핵심이다. 전화번호 형식은 나라마다 다르니 `"ko-KR"` 로 한국 형식을 명시한다. 이 인자를 빼면 한국 번호가 거부될 수 있다.

검사 대상은 문자열이다 — validator의 함수는 문자열을 받는다. `isNumeric("12345")` 처럼 따옴표가 붙어 있다. 숫자처럼 보여도 따옴표로 감싼 문자열을 넘긴다. 웹 요청으로 들어오는 값은 어차피 대부분 문자열이라 자연스럽게 들어맞는다.


```
// 비밀번호 강도 검증
console.log(
  "비밀번호 강도:",
  validator.isStrongPassword("Password123!", {
    minLength: 8,
    minLowercase: 1,
    minUppercase: 1,
    minNumbers: 1,
    minSymbols: 1,
  })
);
```

`isStrongPassword` 는 두 번째 인자로 **기준 객체**를 받는다. 최소 길이 8, 소문자·대문자·숫자·기호 각 1개 이상. 이 조건을 모두 만족해야 `true` 다. `"Password123!"` 은 대문자 P, 소문자 여럿, 숫자 123, 기호 !를 다 갖췄으니 통과한다. 회원가입에서 비밀번호 규칙을 강제할 때 그대로 쓴다.

```
// 문자열 길이 검증
console.log("문자열 길이 검증:", validator.isLength("Hello World", { min: 5, max: 20 }));

// 문자열 정규화
console.log("문자열 정규화:", validator.trim(" Hello World "));
```

`isLength` 는 길이가 범위 안인지 본다(제목·닉네임 길이 제한 등). `trim` 은 검사가 아니라 **변형** 함수다 — 앞뒤 공백을 떼어 깔끔한 문자열로 돌려준다. `validator`에는 이렇게 검사뿐 아니라 값을 다듬는 함수도 섞여 있다.

uuid — 고유 ID 만들기

```
// UUID v4 생성
const uuid = uuidv4();
console.log("UUID v4:", uuid); // 예: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d

// UUID 검증
console.log("UUID 유효성:", validator.isUUID(uuid));
```

`uuidv4()` 를 부를 때마다 `9b1deb4d-3b7d-` 같은 **36자 문자열**이 새로 나온다. 사실상 절대 겹치지 않으므로, 데이터에 고유 ID를 붙일 때 쓴다. 만든 값이 진짜 UUID 형식인지는 `validator`의 `isUUID` 로 되짚어 볼 수 있다 — 두 패키지가 자연스럽게 맞물린다.

```
// UUID를 객체 ID로 사용
const user = {
  id: uuidv4(),
  name: "홍길동",
  email: "hong@example.com",
};
console.log("UUID를 ID로 사용:", user);
```

UUID가 가장 많이 쓰이는 자리다. 데이터를 만들 때 `id` 로 박아 둔다. 앞 장의 인메모리 배열에서는 1, 2, 3 처럼 숫자를 직접 늘려 썼지만, 그 방식은 서버가 여럿이면 같은 번호가 겹친다. UUID는 어디서 만들든 겹치지 않으니, 사용자 ID·세션 ID·트랜잭션 ID·업로드 파일명까지 두루 쓴다. 세션·트랜잭션에 ID를 붙이는 것도 모두 같은 패턴이다.

```
// UUID 타임스탬프 추출 (v1의 경우)
const uuidv1 = require("uuid").v1;
const timeBasedUuid = uuidv1();
console.log("시간 기반 UUID:", timeBasedUuid);
```

v1은 생성된 시각을 바탕으로 만드는 버전이라, 만든 순서가 ID에 묻어난다. 정렬이 필요할 때 쓰지만, 일반적으로는 무작위 기반의 v4를 기본으로 잡으면 된다.

날짜 다루기 (dayjs)

자바스크립트의 기본 `Date` 는 날짜를 더하거나 빼고, 형식을 바꾸고, 차이를 계산하는 일이 번거롭다. **dayjs**는 그 일을 짧고 읽기 좋은 형태로 정리한 가벼운 날짜 라이브러리다.

```
const dayjs = require("dayjs");
require("dayjs/locale/ko"); // 한국어 로케일 추가
const utc = require("dayjs/plugin/utc");
const relativeTimePlugin = require("dayjs/plugin/relativeTime");
// dayjs는 핵심만 가볍게 유지하고, 부가 기능은 플러그인으로 켜다.
dayjs.extend(utc); // .utc() 사용
dayjs.extend(relativeTimePlugin); // .fromNow() 사용
dayjs.locale("ko");
```

위부분에 dayjs의 특징이 그대로 드러난다. dayjs는 **핵심만 가볍게** 유지하고, 부가 기능은 **플러그인으로** 따로 켜서 쓴다.

- `require("dayjs/plugin/utc")` 로 플러그인을 불러온 뒤 `dayjs.extend(utc)` 로 켜야 비로소 `.utc()` 를 쓸 수 있다.
- 마찬가지로 `relativeTime` 플러그인을 켜야 `.fromNow()` 가 동작한다.
- `require("dayjs/locale/ko")` + `dayjs.locale("ko")` 는 요일·상대시간 표현을 한국어로 바꾼다.

플러그인을 켜지 않고 `.utc()` 나 `.fromNow()` 를 부르면 함수가 없다는 에러가 난다. "가벼움을 유지하되 필요한 것만 골라 쓴다"는 게 dayjs의 설계다.

```
// 현재 시간
console.log("현재 시간 (dayjs):", dayjs().format("YYYY-MM-DD HH:mm:ss"));

// 날짜 포매팅
const dateDayjs = dayjs("2024-03-20");
console.log("날짜 포매팅 (dayjs):", dateDayjs.format("YYYY년 MM월 DD일"));
```

`dayjs()` 는 현재 시각을, `dayjs("2024-03-20")` 은 특정 날짜를 dayjs 객체로 만든다. `.format(.)` 은 그 객체를 원하는 문자열 모양으로 찍는다. YYYY (연 4자리), MM (월 2자리), DD (일), HH:mm:ss (시:분:초) 같은 토큰을 조합한다. "YYYY년 MM월 DD일" 처럼 토큰 사이에 한글을 끼워도 그대로 남는다.

```
// 시간 더하기/빼기
console.log("일주일 후 (dayjs):", dayjs().add(7, "day").format("YYYY-MM-DD"));

// 시간 차이 계산
const startDayjs = dayjs("2024-01-01");
const endDayjs = dayjs("2024-12-31");
console.log("날짜 차이 (dayjs):", endDayjs.diff(startDayjs, "day"), "일");
```

- `.add(7, "day")` — 7일 뒤를 구한다. "day" 자리에 "month", "year" 등을 넣어 단위를 바꾼다. 빼기는 `.subtract(.)` 다.
- `.diff(다른날짜, "day")` — 두 날짜의 차이를 일 단위로 계산한다. 여기서는 1년 차이로 365가 나온다. "가입한 지 며칠 됐나", "마감까지 며칠 남았나" 같은 계산이 한 줄로 끝난다.

```
// 상대적 시간 표시 (relativeTime 플러그인)
console.log("상대적 시간 (dayjs):", dayjs("2024-03-15").fromNow());

// 시간대 변환 (utc 플러그인)
console.log("UTC 시간 (dayjs):", dayjs().utc().format());
```

이 두 줄이 앞에서 켜 플러그인의 결과다.

- `.fromNow()` — "3개월 전" 처럼 **지금 기준 상대 시간**을 한국어로 돌려준다(`relativeTime` 플러그인 + 한국어 로케일). SNS의 "방금 전", "2시간 전" 표시가 이것이다.
- `.utc()` — 현지 시각을 세계 표준시(UTC)로 바꾼다(`utc` 플러그인). 서버마다 시간대가 다를 때, 저장은 UTC로 통일하는 게 흔한 방식이다.

```
// 날짜 유효성 검사
console.log("날짜 유효성 (dayjs):", dayjs("2024-13-45").isValid()); // false

// 날짜 비교
console.log("날짜 비교 (dayjs):", dayjs("2024-03-20").isAfter("2024-03-19")); // true
```

- `.isValid()` — 말이 되는 날짜인지 본다. `2024-13-45` (13월 45일)는 존재하지 않으니 `false` 다. 사용자가 입력한 날짜를 받아들이기 전에 거른다.
- `.isAfter(.)` / `.isBefore(.)` / `.isSame(.)` — 두 날짜의 앞뒤를 비교해 `true` / `false` 를 돌려준다.

```
// 날짜 범위 생성
const start = dayjs("2024-01-01");
const end = dayjs("2024-01-05");
const range = [];
for (
  let date = start;
  date.isBefore(end) || date.isSame(end, "day");
  date = date.add(1, "day")
) {
  range.push(date.format("YYYY-MM-DD"));
}
console.log("날짜 범위 (dayjs):", range);
```

마지막 예제는 지금까지의 기능을 한데 엮는다. 1월 1일 부터 1월 5일 까지 하루씩 늘려 가며 날짜 목록을 만든다. 반복 조건 `date.isBefore(end) || date.isSame(end, "day")` 는 "끝날 이전이거나, 끝날과 같은 날이면 계속"이라는 뜻 — 끝날까지 포함한다. 매번 `date.add(1, "day")` 로 하루를 더한다. 결과는 `["2024-01-01", ..., "2024-01-05"]` 다. 달력 만들거나 기간 통계에서 쓰는 패턴이다.

흔한 실수

- `npm install` 없이 실행한다. → `Cannot find module 'dotenv'` 같은 에러. `node_modules/` 가 없으면 `require` 가 패키지를 못 찾는다. 먼저 `npm install`.
- `.env` 를 안 만들고 `dotenv` 예제를 돌린다. → `process.env.PORT` 가 `undefined`. `cp .env.example .env` 를 빼먹은 경우다.
- `require("dotenv").config()` 를 다른 코드 아래에 둔다. → 그 위에서 `process.env` 를 읽으면 값이 아직 안 들어와 있다. 항상 파일 맨 위.
- `dayjs` 플러그인을 안 켜고 `.fromNow()` 나 `.utc()` 를 부른다. → 함수가 없다는 에러. `dayjs.extend(.)` 로 먼저 켜다.
- `validator` 함수에 숫자나 객체를 넘긴다. → `validator`는 문자열을 받는다. `isNumeric(12345)` 가 아니라 `isNumeric("12345")`.

- `.env` 를 깃에 커밋한다. → 키와 비밀번호가 그대로 노출된다. `.gitignore` 에 `.env` 를 넣고 `.env.example` 만 올린다.

직접 해보기

1. `.env` 에 새 키 `APP_NAME=내앱` 을 추가하고, `process.env.APP_NAME` 을 콘솔에 찍어 보자. `dotenv`가 자동으로 읽어 오는지 확인.
2. `winston`의 첫 로거를 `level: "debug"` 로 바꿔 실행해 보자. 가려졌던 `logger.debug(. )` 줄이 이제 출력되는지 본다. 반대로 `level: "warn"` 으로 두면 `info` 까지 사라지는 것도 확인.
3. `validator`에 일부러 잘못된 값을 넣어 보자. `validator.isEmail("not-an-email")` 이 `false` 를, 약한 비밀번호 `"abc"` 가 `isStrongPassword` 에서 `false` 를 돌려주는지 확인한다.

정리

- 외부 모듈은 `npm install <패키지>` 로 먼저 내려받아야 `require` 로 쓸 수 있다. 그 명령이 `package.json` 의 `dependencies` 를 채운다.
- `dotenv`는 설정값을 `.env` 로 빼내 `process.env` 로 읽게 한다. `.env` 는 깃에서 빼고 `.env.example` 만 공유한다.
- `winston`은 로그를 레벨로 나눠 콘솔과 파일에 동시에 남긴다. `level` 로 출력 문턱을, `transports` 로 출력 대상을 정한다.
- `validator`는 이메일·URL·비밀번호 등 문자열 형식을 검사하고, `uuid`는 겹치지 않는 고유 ID를 만든다.
- `dayjs`는 날짜를 더하고·빼고·비교하고·포맷한다. 부가 기능(`.utc()` , `.fromNow()`)은 플러그인을 `extend` 로 커야 동작한다.

다음 장(**04 · Express 입문**)에서는 이렇게 가져다 쓰는 패키지 가운데 가장 중요한 하나, 웹 프레임워크 Express로 첫 JSON API 서버를 띄운다.